

C Language

Cheat Sheet Study Book

A compact, elegant reference for syntax, vocabulary, core theory, patterns, and dangerous edge cases

Designed for learners who want to understand C, not only memorize C. Includes clear examples, practical notes, undefined-behavior warnings, and build/debug vocabulary.

What this book gives you	How to use it
Vocabulary: translation unit, object, lvalue, linkage, storage duration, pointer, array decay, undefined behavior.	Read Chapter 1 first, then keep the tables open while coding.
Syntax patterns: declarations, functions, structs, pointers, memory allocation, files, preprocessor.	Copy the examples and compile them with warnings enabled.
Edge cases: signed overflow, dangling pointers, buffer overflow, format string mismatches, macro traps.	Treat the red warning boxes as rules you verify during code review.

Table of Contents

This book is organized as a practical study reference. Each chapter introduces vocabulary, syntax, examples, and common mistakes.

1. The C Mental Model
2. Build, Translation, and Program Shape
3. Types, Literals, and Conversions
4. Operators and Expressions
5. Control Flow
6. Functions and Declarations
7. Pointers
8. Arrays and Strings
9. Structs, Unions, Enums, and typedef
10. Dynamic Memory
11. The Preprocessor
12. Storage Duration, Scope, and Linkage
13. Input/Output and Files
14. Error Handling and Defensive C
15. Debugging, Tooling, and Build Habits
16. Edge-Case Field Guide
17. Quick Reference Tables
18. Glossary and Practice

1. The C Mental Model

C is a small language that gives you direct access to memory and hardware-like concepts. It does not protect you automatically from many invalid operations. This is the source of both its power and its danger.

Concept	Meaning	Why it matters
Object	A region of storage that can hold a value.	Variables and allocated memory are objects; pointers refer to objects.
Value	The data represented by an object or expression.	A value may be copied, converted, compared, or passed to a function.
Type	A rule set describing size, representation, and valid operations.	The compiler uses types to check expressions and choose conversions.
Expression	Code that computes a value, may also have side effects.	Assignments and function calls are expressions in C.
Side effect	A change to program state: modifying an object, writing output, volatile access.	Order of evaluation matters when side effects interact.
Undefined behavior	A program action for which the C standard imposes no requirements.	The compiler may optimize assuming it never happens.
Implementation-defined	Behavior chosen and documented by the compiler/platform.	Example: size of int, signedness of plain char.
Unspecified	One of several valid behaviors, not necessarily documented.	Example: order of evaluating many function arguments.

Core idea

C is not primarily object-oriented or memory-safe. Think in terms of storage, addresses, types, lifetimes, and ownership conventions.

A minimal program

```
#include <stdio.h>

int main(void)
{
    printf("Hello, C!\n");
    return 0;
}
```

The function `main` is the normal entry point. Returning 0 usually means successful execution. Header files such as `<stdio.h>` provide declarations for library functions.

2. Build, Translation, and Program Shape

A C program is usually split into source files .c and header files .h. Each source file is compiled into an object file, then object files and libraries are linked into an executable.

Stage	Typical command	What happens
Preprocess	gcc -E main.c	Handles #include, #define, conditional compilation.
Compile	gcc -S main.c	Turns preprocessed C into assembly.
Assemble	gcc -c main.c	Creates object file main.o.
Link	gcc main.o util.o -o app	Resolves external symbols and produces executable.

Recommended compiler flags while learning

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wshadow -Wconversion -g -O0 main.c -o app
```

Sanitizers for runtime checks during development:

```
gcc -std=c17 -Wall -Wextra -g -fsanitize=address,undefined main.c -o app
```

Vocabulary

A translation unit is one preprocessed source file. A declaration introduces a name and type. A definition creates the object or function body. A function may be declared many times but defined once in the final linked program.

Header/source pattern

```
/* math_utils.h */
#ifndef MATH_UTILS_H
#define MATH_UTILS_H
int add(int a, int b);    /* declaration */
#endif

/* math_utils.c */
#include "math_utils.h"
int add(int a, int b)    /* definition */
{
    return a + b;
}

/* main.c */
#include <stdio.h>
#include "math_utils.h"
int main(void)
{
    printf("%d\n", add(2, 3));
}
```

3. Types, Literals, and Conversions

C has integer types, floating types, character types, pointers, arrays, structures, unions, enumerations, and function types. The exact size of several integer types depends on platform and compiler.

Category	Examples	Notes
Integer	char, short, int, long, long long	Can be signed or unsigned. Use for exact-width types.
Exact-width	int32_t, uint64_t	Available when the platform supports exact sizes.
Size/count	size_t	Unsigned type returned by sizeof; used for object sizes and indexes.
Pointer difference	ptrdiff_t	Signed type for subtracting pointers within the same array.
Floating	float, double, long double	Floating arithmetic has rounding and special values.
Boolean	bool from	Historically C used 0 for false and nonzero for true.

Literals

Literal	Meaning	Example
42	decimal int	int x = 42;
0x2A	hexadecimal	int mask = 0xff;
052	octal	Beware: leading zero means base 8.
42u	unsigned	unsigned n = 42u;
3.14	double	double pi = 3.14;
3.14f	float	float f = 3.14f;
'A'	character constant	In C, character constants have type int.
"abc"	string literal	An array of char with a terminating '\0'.

Conversions that surprise beginners

- Integer promotion: small integer types such as char and short are usually promoted to int in expressions.
- Usual arithmetic conversions: when signed and unsigned types mix, the signed value may convert to unsigned.
- Floating-to-integer conversion truncates toward zero; overflow is undefined if the value cannot fit.
- Assignment converts the right-hand value to the type of the left-hand object.

```
#include <stdio.h>
int main(void)
{
    int a = -1;
    unsigned b = 1;
    printf("%d\n", a < b); /* often prints 0: a converts to unsigned */
}
```

Safer habit

Do not casually mix signed and unsigned values. For sizes, use `size_t`. For arithmetic that can go negative, use a signed type and check bounds before converting.

4. Operators and Expressions

Group	Operators	Notes
Arithmetic	+ - * / %	% works with integers. Division by zero is undefined.
Comparison	== != < <= > >=	Result is int: 0 or 1.
Logical	&& !	&& and short-circuit.
Bitwise	& ^ ~ << >>	Use for flags, masks, embedded registers.
Assignment	= += -= *= /= %= ...	Assignment expression has the assigned value.
Pointer	& * -> .	& takes address; * dereferences; -> accesses through pointer.
Size	sizeof	Usually compile-time for ordinary objects; returns size_t.
Conditional	cond ? a : b	Expression-level if/else.
Comma	,	Evaluates left then right; rarely needed.

Precedence rule of thumb

Do not memorize the entire precedence table at first. Memorize the dangerous zones and use parentheses for clarity.

Expression	Meaning	Safer writing
*p++	*(p++): use pointed value, then advance pointer	int v = *p; p++;
*++p	*(++p): advance pointer, then use pointed value	p++; int v = *p;
(*p)++	increment the object pointed to	(*p) += 1;
a & b == 0	a & (b == 0), because == binds tighter than &	(a & b) == 0

Undefined behavior warning

Do not modify the same scalar object more than once between sequence points or in unsequenced evaluation. Avoid expressions like `i = i++ + 1`. Split such code into multiple statements.

```
int flags = 0;
#define FLAG_READY (1u << 0)
#define FLAG_ERROR (1u << 1)

flags |= FLAG_READY;          /* set bit */
flags &= ~FLAG_ERROR;         /* clear bit */
if ((flags & FLAG_READY) != 0) {
    /* ready */
}
```

5. Control Flow

Construct	Pattern	Use
if / else	if (condition) { ... } else { ... }	Branching by condition.
while	while (condition) { ... }	Loop while condition remains true.
do while	do { ... } while (condition);	Loop body runs at least once.
for	for (init; condition; step) { ... }	Counting, scanning, iteration.
switch	switch (expr) { case X: ...; break; }	Multi-way branch on integer-like expression.
break	break;	Exit nearest loop or switch.
continue	continue;	Skip to next loop iteration.
goto	goto cleanup;	Rare; useful for centralized cleanup in C.

Loop patterns

```

/* Count through an array safely */
for (size_t i = 0; i < count; ++i) {
    process(values[i]);
}

/* Read until EOF */
int ch;
while ((ch = getchar()) != EOF) {
    putchar(ch);
}

```

switch fall-through

```

switch (command) {
case 's':
    start();
    break;
case 'q':
    quit();
    break;
default:
    fprintf(stderr, "unknown command\n");
    break;
}

```

Edge case

Falling through from one case into the next is allowed unless you use break. This is sometimes intentional, but should be commented clearly because it is also a common bug.

6. Functions and Declarations

Functions group behavior and define an interface. C passes arguments by value. To let a function modify an outside object, pass a pointer to that object.

```
#include <stdbool.h>

bool divide(int a, int b, int *out_result)
{
    if (b == 0 || out_result == NULL) {
        return false;
    }
    *out_result = a / b;
    return true;
}
```

Term	Meaning
Prototype	A declaration that includes parameter types: <code>int f(int x);</code>
Definition	The function body: <code>int f(int x) { return x + 1; }</code>
Parameter	The named object inside the function.
Argument	The expression passed by the caller.
Return type	The type of the expression returned to the caller.
void	No return value or no parameters when written as <code>f(void)</code> .

Function pointer basics

```
int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }

typedef int (*binary_op_fn)(int, int);

int apply(binary_op_fn op, int x, int y)
{
    return op(x, y);
}

int result = apply(add, 2, 3);
```

Best practice

Prefer declaring functions before use, usually in a header. Since modern C, relying on implicit declarations is invalid or at least a serious compiler diagnostic.

7. Pointers

A pointer stores the address of an object or function. Pointers are typed: an `int *` points to an `int`, a `char *` points to a `char`, and so on.

```
int x = 10;
int *p = &x;    /* p stores the address of x */
printf("%d\n", *p); /* *p reads x */
*p = 20;        /* modifies x through p */
```

Syntax	Meaning
<code>int *p;</code>	<code>p</code> is a pointer to <code>int</code> .
<code>&x;</code>	address of <code>x</code> .
<code>*p</code>	object pointed to by <code>p</code> ; dereference.
<code>p == NULL</code>	<code>p</code> points to no object.
<code>p + 1</code>	address of the next <code>int</code> object, not next byte.
<code>void *</code>	generic object pointer; must convert before dereference.
<code>const int *p</code>	pointer to <code>const int</code> ; cannot modify <code>*p</code> through <code>p</code> .
<code>int * const p</code>	<code>const</code> pointer to <code>int</code> ; <code>p</code> cannot be reassigned.

const pointer vocabulary

```
const int *a;    /* pointer to const int: *a cannot be changed through a */
int const *b;    /* same as above */
int * const c = &x; /* const pointer: c cannot point elsewhere */
const int * const d = &x; /* both pointer and pointed object are const via d */
```

Danger

Dereferencing a `NULL`, uninitialized, dangling, or invalid pointer is undefined behavior. A pointer value can look like a number, but only valid object/function addresses may be used.

8. Arrays and Strings

An array is a contiguous sequence of elements of the same type. In most expressions, an array name converts, or decays, to a pointer to its first element. Important exception: `sizeof` array gives the full array size when the actual array is in scope.

```
int a[4] = {10, 20, 30, 40};
size_t n = sizeof a / sizeof a[0]; /* only works where a is an actual array */

for (size_t i = 0; i < n; ++i) {
    printf("%d\n", a[i]);
}
```

Array decay example

```
void print_count(int values[]) /* actually int *values */
{
    /* sizeof values is sizeof(int *), not the array size */
}

void print_array(const int *values, size_t count)
{
    for (size_t i = 0; i < count; ++i) {
        printf("%d\n", values[i]);
    }
}
```

C strings

A C string is a sequence of `char` ending with a null character `'\0'`. The standard string functions rely on that terminator.

Function	Header	Purpose	Common trap
<code>strlen</code>		Count characters before <code>'\0'</code> .	Does not include terminator.
<code>strcpy</code>		Copy string including terminator.	Unsafe if destination too small.
<code>strncpy</code>		Copy limited count.	May not null-terminate.
<code>strcmp</code>		Lexicographic compare.	Returns 0 when equal.
<code>snprintf</code>		Formatted write with buffer size.	Check return value for truncation.

```
char name[16];
int written = snprintf(name, sizeof name, "%s", input);
if (written < 0 || (size_t)written >= sizeof name) {
    /* output was truncated or formatting failed */
}
```

9. Structs, Unions, Enums, and typedef

struct

A structure groups named fields into one object. Use the dot operator for structure objects and arrow for pointers to structures.

```
struct Point {
    int x;
    int y;
};

struct Point p = { .x = 10, .y = 20 };
struct Point *pp = &p;
printf("%d %d\n", p.x, pp->y);
```

typedef

```
typedef struct {
    const char *host;
    int port;
} BrokerConfig;

BrokerConfig cfg = { .host = "localhost", .port = 1883 };
```

enum

```
enum Status {
    STATUS_OK = 0,
    STATUS_TIMEOUT = 1,
    STATUS_ERROR = 2
};
```

union

A union stores different members in the same memory. Only one member is valid at a time unless special representation rules apply. A tagged union stores an enum tag plus a union payload.

```
enum ValueKind { VAL_INT, VAL_DOUBLE };

struct Value {
    enum ValueKind kind;
    union {
        int i;
        double d;
    } as;
};
```

Layout note

Structures may include padding bytes for alignment. Do not assume a struct has no gaps unless you control packing carefully and understand the ABI/hardware consequences.

10. Dynamic Memory

Dynamic allocation obtains storage from the heap at runtime. The programmer is responsible for freeing it exactly once and not using it after it has been freed.

Function	Header	Purpose	Notes
<code>malloc(size)</code>		Allocate uninitialized bytes.	Returns NULL on failure.
<code>calloc(n, size)</code>		Allocate and zero n elements.	Can help avoid multiplication mistakes in intent, but overflow still matters.
<code>realloc(ptr, size)</code>		Resize allocation.	Use a temporary pointer to avoid losing original on failure.
<code>free(ptr)</code>		Release allocation.	<code>free(NULL)</code> is safe.

```
int *make_array(size_t n)
{
    if (n > SIZE_MAX / sizeof(int)) {
        return NULL; /* overflow guard */
    }
    int *p = malloc(n * sizeof *p);
    if (p == NULL) {
        return NULL;
    }
    for (size_t i = 0; i < n; ++i) {
        p[i] = 0;
    }
    return p;
}

int *values = make_array(100);
if (values != NULL) {
    /* use values */
    free(values);
    values = NULL;
}
```

Safe realloc pattern

```
int *tmp = realloc(values, new_count * sizeof *values);
if (tmp == NULL) {
    /* values is still valid; handle failure */
} else {
    values = tmp;
}
```

Memory ownership

Every dynamically allocated object should have an obvious owner responsible for calling free. Document ownership in function names or comments: create/destroy, init/cleanup, borrow/own.

11. The Preprocessor

The preprocessor runs before compilation. It performs textual inclusion and macro expansion. It does not understand C types the way the compiler does.

Directive	Use
#include	Include system header.
#include "my.h"	Include project header.
#define NAME value	Object-like macro.
#define F(x) ((x) + 1)	Function-like macro.
#if / #ifdef / #ifndef	Conditional compilation.
#error message	Force compile-time error.
#pragma	Implementation-specific instruction.

Macro hygiene

```
#define SQUARE_BAD(x) x * x
#define SQUARE(x) ((x) * (x))

int a = SQUARE_BAD(1 + 2); /* expands to 1 + 2 * 1 + 2 */
int b = SQUARE(1 + 2);    /* expands to ((1 + 2) * (1 + 2)) */
```

Macro danger

Even a parenthesized macro may evaluate an argument more than once: `SQUARE(i++)` increments `i` twice. Prefer inline functions when type-generic behavior is not required.

Include guards

```
#ifndef CONFIG_H
#define CONFIG_H

#define DEFAULT_PORT 1883
struct Config { const char *host; int port; };

#endif /* CONFIG_H */
```

12. Storage Duration, Scope, and Linkage

Concept	Options	Meaning
Scope	block, function, file	Where a name can be used in source code.
Storage duration	automatic, static, allocated, thread	How long the object exists.
Linkage	none, internal, external	Whether declarations in different scopes/translation units refer to the same entity.

Storage duration examples

```

int global_count;           /* static storage duration, external linkage */
static int file_counter;    /* static storage duration, internal linkage */

void f(void)
{
    int local = 0;          /* automatic storage duration */
    static int calls = 0;    /* static storage duration, block scope */
    ++calls;
}

```

extern and static

```

/* config.h */
extern int g_log_level;    /* declaration, defined elsewhere */

/* config.c */
int g_log_level = 2;       /* definition */

/* helper.c */
static void helper(void)   /* visible only in this translation unit */
{
}

```

Rule of thumb

Use static for private file-level functions and variables. Expose only the API that other source files truly need.

13. Input/Output and Files

The standard I/O library uses FILE * streams. Always check file opening and writing errors in production code.

Function	Purpose	Notes
printf	Print formatted output to stdout.	Format specifier must match argument type.
fprintf	Print to a specific stream.	Use stderr for errors.
snprintf	Format into buffer with size limit.	Preferred over sprintf.
fopen	Open file.	Returns NULL on failure.
fgets	Read a line into buffer.	Keeps newline if it fits.
fread/fwrite	Binary input/output.	Check element count returned.
fclose	Close stream.	Can fail, especially when flushing output.

Format specifiers

Type	printf	scanf	Notes
int	%d	%d	scanf needs int *.
unsigned	%u	%u	Use matching type.
long	%ld	%ld	l length modifier.
size_t	%zu	%zu	C99 and later.
double	%f	%lf	printf promotes float to double; scanf differs.
char * string	%s	%s	Limit scanf width to avoid overflow.
void * pointer	%p	-	Cast to void * for printf.

```
FILE *fp = fopen("data.txt", "r");
if (fp == NULL) {
    perror("fopen");
    return 1;
}

char line[256];
while (fgets(line, sizeof line, fp) != NULL) {
    fputs(line, stdout);
}

if (fclose(fp) != 0) {
    perror("fclose");
}
```

14. Error Handling and Defensive C

C does not have exceptions. Error handling is usually explicit: return codes, NULL pointers, errno, output parameters, and cleanup paths.

Pattern	Example	When useful
Return status	<code>bool parse(...);</code>	Operation can fail but also returns output through pointer.
Return pointer or NULL	<code>malloc, fopen</code>	Object creation or lookup.
Return negative errno-style	<code>return -EINVAL;</code>	Low-level/system-style APIs.
errno	<code>perror, strerror(errno)</code>	Library/system calls that document errno use.

Centralized cleanup

```
int process_file(const char *path)
{
    FILE *fp = NULL;
    char *buffer = NULL;
    int rc = -1;

    fp = fopen(path, "rb");
    if (fp == NULL) goto cleanup;

    buffer = malloc(4096);
    if (buffer == NULL) goto cleanup;

    /* work */
    rc = 0;

cleanup:
    free(buffer);
    if (fp != NULL) fclose(fp);
    return rc;
}
```

Defensive checklist

Check every allocation, file open, parse result, array bound, string length, and external input. C code often fails not because syntax is hard, but because assumptions were not checked.

15. Debugging, Tooling, and Build Habits

A professional C workflow uses warnings, static analysis, runtime sanitizers, debuggers, and a build system.

Tool/habit	Purpose	Example
Warnings	Catch suspicious code at compile time.	-Wall -Wextra -Wpedantic
Debug symbols	Let debugger map machine code to source.	-g
Sanitizers	Find memory and undefined behavior bugs at runtime.	-fsanitize=address,undefined
gdb/lldb	Step through code and inspect variables.	gdb ./app
valgrind	Find leaks/invalid memory on supported platforms.	valgrind ./app
clang-tidy/cppcheck	Static analysis.	clang-tidy *.c --
Make/CMake	Repeatable builds.	make, cmake --build

Tiny Makefile

```
CC := gcc
CFLAGS := -std=c17 -Wall -Wextra -Wpedantic -g

app: main.o util.o
    $(CC) $(CFLAGS) $^ -o $@

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -f app *.o
```

gdb essentials

```
gdb ./app
(gdb) break main
(gdb) run
(gdb) next
(gdb) step
(gdb) print variable
(gdb) backtrace
(gdb) quit
```

16. Edge-Case Field Guide

Edge case	Bad example	Safer approach
Signed overflow	<code>int x = INT_MAX + 1;</code>	Check before arithmetic or use wider type.
Integer division	<code>5 / 2</code> gives 2	Cast intentionally if floating result is needed.
Uninitialized read	<code>int x; printf("%d", x);</code>	Initialize variables before use.
Dangling pointer	<code>return &local;</code>	Return by value or allocate with clear ownership.
Array out of bounds	<code>a[n]</code> when <code>n == count</code>	Use <code>< count</code> and pass count explicitly.
String not terminated	<code>strncpy</code> may omit <code>'\0'</code>	Use <code>sprintf</code> or manually terminate.
Format mismatch	<code>printf("%d", size_t_value)</code>	Use <code>%zu</code> for <code>size_t</code> .
Macro double eval	<code>MAX(i++, j++)</code>	Use inline functions or avoid side effects.
Strict aliasing	Access object through incompatible pointer	Use correct type or <code>memcpy</code> for representation.
Use after free	<code>free(p); *p = 1;</code>	Set pointer NULL and design ownership.

Common declaration confusion

```

int *a, b;      /* a is pointer to int, b is int */
int (*fp)(int); /* fp is pointer to function taking int returning int */
int *arr[10];   /* array of 10 pointers to int */
int (*pa)[10];  /* pointer to array of 10 int */

```

Reading declarations

Start at the identifier and spiral outward: `arr` is an `array[10]` of pointer to `int`; `pa` is a pointer to `array[10]` of `int`.

17. Quick Reference Tables

Headers you will use often

Header	Provides
	printf, fprintf, snprintf, FILE, fopen, fgets, fread, fwrite, fclose
	malloc, calloc, realloc, free, exit, strtol, qsort, bsearch
	strlen, strcmp, memcpy, memmove, memset, strerror
	int32_t, uint64_t, uintptr_t and other integer types
	bool, true, false
	size_t, ptrdiff_t, NULL, offsetof
	errno and error constants
	isalpha, isdigit, toupper; pass unsigned char or EOF
	sqrt, sin, cos; often link with -lm on Unix-like systems

Pointer and memory functions

Function	Use	Trap
memcpy(dst, src, n)	Copy n bytes between non-overlapping objects.	Overlapping regions: use memmove.
memmove(dst, src, n)	Copy n bytes safely even if overlap.	Still needs valid buffers.
memset(p, 0, n)	Set n bytes.	Not type-aware; not for all representations.
memcmp(a, b, n)	Compare bytes.	Not semantic compare for structs with padding.

Declaration mini-cheat sheet

Declaration	Meaning
int x;	x is an int.
int *p;	p points to int.
const char *s;	s points to read-only chars through s.
char buf[128];	buf is an array of 128 char.
int f(int);	f is a function taking int returning int.
int (*pf)(int);	pf is a pointer to such a function.
struct Node *next;	next points to a struct Node.

18. Glossary and Practice

Glossary

Term	Definition
ABI	Application Binary Interface: calling conventions, layout, register use, binary compatibility rules.
Alignment	Requirement that an object address be a multiple of a certain value.
Array decay	Automatic conversion of an array expression to a pointer to its first element in most contexts.
Automatic storage	Storage for local block variables, usually on the stack, ending when the block exits.
Dangling pointer	Pointer to an object whose lifetime has ended.
Heap	Common name for dynamically allocated storage managed by malloc/free.
Lvalue	Expression that designates an object; often can appear on the left of assignment.
Null pointer	Pointer value guaranteed not to point to any object or function.
Object file	Compiled machine code plus relocation/symbol information before final link.
Padding	Unused bytes inserted for alignment or layout.
Translation unit	A source file after preprocessing.
Undefined behavior	A program construct for which no behavior is guaranteed by the C standard.

Practice exercises

- 1 Write a program that reads lines with fgets and counts how many are longer than 80 characters.
- 2 Create a function bool parse_port(const char *s, int *out_port) using strtol and range checks.
- 3 Implement a dynamic int array with create, push, get, and destroy functions.
- 4 Write a struct-based configuration object and pass it as const pointer to three functions.
- 5 Write a small program that demonstrates the difference between sizeof array and sizeof pointer.
- 6 Create a Makefile that builds main.c plus two modules and supports make clean.
- 7 Compile one intentionally unsafe program with AddressSanitizer and observe the report.

Final review checklist

- Can you explain declaration vs definition?
- Can you explain why arrays usually need a separate count parameter?
- Can you identify ownership of every malloc result?
- Can you explain the difference between const int * and int * const?
- Can you explain why printf format mismatches are dangerous?
- Can you compile with warnings and understand every warning before running?

Study method

For each table in this book, write a tiny compilable C program that demonstrates the concept. C becomes much clearer when vocabulary, syntax, and runtime behavior are connected.